

State Pattern mit Spring Boot

Retro Kit Order Workflow als Bestellsystem

Präsentationsgrundlage

IT-Kolleg Projektarbeit

16. Mai 2026

- Eine Bestellung durchläuft mehrere fachliche Zustände.
- Im `without-pattern-Branch` liegt die Logik zentral in `KitOrderService`.
- Aktionen wie Bezahlen, Verpacken oder Versenden werden dort mit Fallunterscheidungen geprüft.
- Mit jedem neuen Status wächst die Service-Klasse weiter.

Zentrale Frage: Wie kapseln wir zustandsabhängiges Verhalten sauber und erweiterbar?

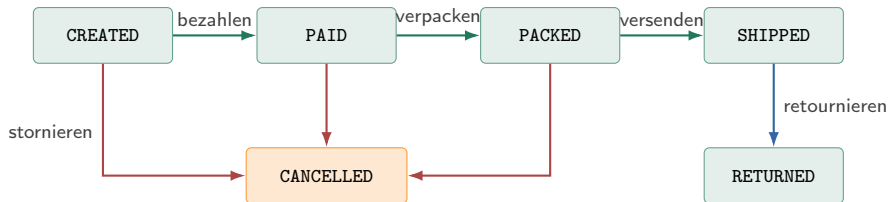
- Eigenständiges Spring-Boot-Projekt im Ordner `state_pattern_kit_orders`
- Java 21, Spring Boot 3.3.5, Maven, Spring Web, Spring Data JPA, H2
- Fachdomäne: Verwaltung von Retro-Fußballtrikot-Bestellungen
- Branches:
 - `without-pattern`: funktionierende Ausgangslösung ohne gezielten Pattern-Einsatz
 - `with-pattern`: refaktorierte State-Pattern-Lösung

Grundidee

Das Verhalten eines Objekts hängt vom aktuellen Zustand ab. Statt großer `if-` oder `switch`-Blöcke wird jeder Zustand als eigene Klasse modelliert.

- Gemeinsames Interface für zustandsabhängige Aktionen
- Eigene Klasse pro Status
- Übergangslogik liegt dort, wo sie fachlich hingehört
- Der zentrale Service orchestriert nur noch

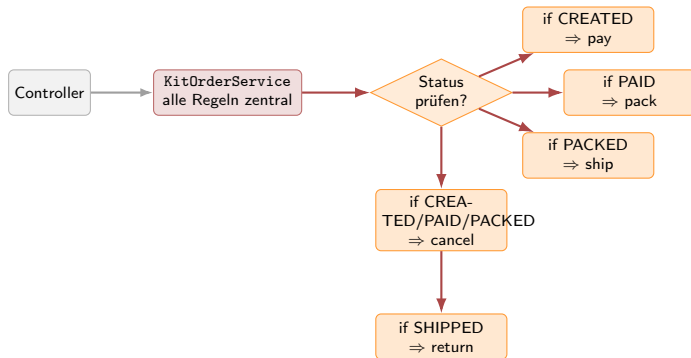
Fachlicher Bestellablauf



Sinnvolle Erweiterung

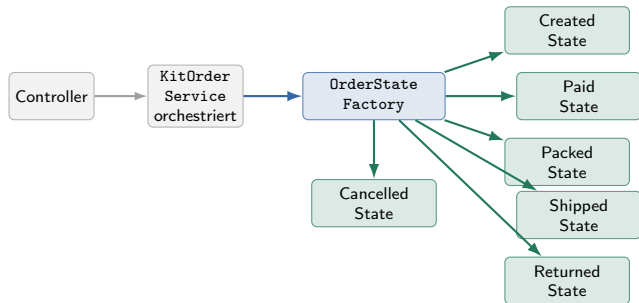
Der Zustand **RETURNED** eignet sich gut, um den Unterschied zwischen **without-pattern** und **with-pattern** zu demonstrieren.

without-pattern: Zentrale Logik



- Ein Service kennt alle Status und alle Sonderfälle.
- Neue Übergänge landen wieder in derselben Klasse.
- **Engpass:** jede Erweiterung vergrößert den zentralen Entscheidungsknoten.
- Das sieht in der Grafik wie ein Bottleneck aus und ist genau das Problem dieser Variante.

with-pattern: Verteilte Architektur



- Der Service entscheidet nicht mehr alles selbst.
- Die Factory liefert die passende Zustandsklasse.
- Fachregeln sind auf mehrere klar benannte Klassen verteilt.
- **Vorteil:** neue Zustände werden lokal ergänzt statt zentral gestaut.

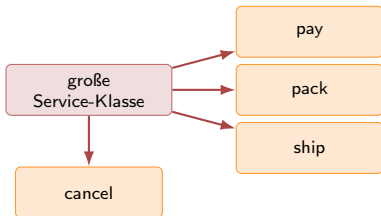
- `POST /api/orders`
- `GET /api/orders`
- `GET /api/orders/{orderId}`
- `GET /api/orders/status/{status}`
- `POST /api/orders/{orderId}/pay`
- `POST /api/orders/{orderId}/pack`
- `POST /api/orders/{orderId}/ship`
- `POST /api/orders/{orderId}/cancel`
- `POST /api/orders/{orderId}/return`

- Mehr als 6 Tests sind vorhanden
- Integrationstests prüfen API und Statuswechsel
- Unit-Tests prüfen im with-pattern-Branch die Zustandsklassen direkt
- Validiert wurden beide Branches mit `./mvnw test`

Beobachtung

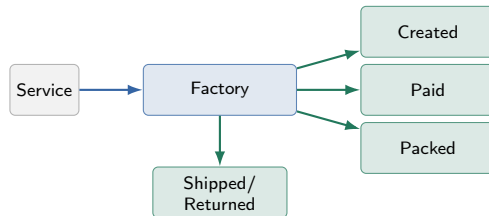
Im with-pattern-Branch werden Tests fachlich klarer, weil Übergänge gezielt auf einzelnen State-Klassen überprüft werden können.

without-pattern



- Engpass in einer Klasse
- Änderung an zentralem Code
- schlechtere grafische Struktur

with-pattern



- verteilte Verantwortung
- lokale Erweiterungen
- klarere Test- und Code-Struktur

- KI half beim Verstehen des State Patterns.
- KI lieferte Vorschläge für Refactoring-Schritte und Testideen.
- KI war hilfreich für die Dokumentation des Branch-Vergleichs.

Wichtig

Die Vorschläge wurden fachlich geprüft und angepasst, besonders bei Zustandsübergängen, API-Design, Fehlerbehandlung und Testabdeckung.

Demo-Ablauf in der Präsentation

- ➊ without-pattern-Branch zeigen
- ➋ zentrale Logik in KitOrderService kurz erklären
- ➌ auf with-pattern wechseln
- ➍ OrderStateFactory und die State-Klassen zeigen
- ➎ Request-Folge demonstrieren:
 - Bestellung anlegen
 - bezahlen
 - verpacken
 - versenden
 - retournieren

- Das State Pattern ist sinnvoll für Objekte mit klaren Lebenszyklen.
- Im Bestellsystem reduziert es zentrale Fallunterscheidungen deutlich.
- Die Erweiterung RETURNED lässt sich im with-pattern-Branch sauberer umsetzen.
- Nachteil: mehr Klassen und etwas höherer Strukturaufwand.

Fragen?